

Requirements Specification

Ready Player Pi

Attachment 1.2

Aarhus University

Department of Electrical & Computer Engineering

Bachelor of Engineering in Software technology



AARHUS UNIVERSITET

TABLE OF CONTENTS

Requirements Specification Ready Player Pi	1
Version history.....	3
Introduction	3
1. System Description	5
2. Functional requirements	10
2.1 Actor descriptions	10
2.2 Actors-context diagram.....	10
2.3 Use Case Diagram.....	11
2.4 Use Case Descriptions	11
2.5 Use Case 1: Application set-up.....	12
2.6 Use Case 2: Start Game Room.....	Error! Bookmark not defined.
2.7 Use Case 3: Play Gyroscope Game “GyroPi”	13
2.8 Use Case 4: Play Note Game “NotePi”	14
3. Project Scope	16
4. Non-Functional Requirements	18

Version history

Version	Date	Description
1.0		

Word Explanation

Name	Communicator Player
Alternative references	CP
Type	Primary
Description	The player that during gameplay receives crucial feedback from the game. Communicator Player needs to communicate this feedback orally and guide Performer Player in order to win the game. The role of Communicator Player is assigned randomly to either Host Player or Quest Player.

Name	Performer Player
Alternative references	PP
Type	Primary
Description	The player that - via feedback received orally by Communicator Player – performs the actions to win the game. The role of Performer Player is assigned randomly to either Host Player or Guest Player.

Introduction

This requirements specification describes the gameflow, gamestates and roles, as well as the functional and non-functional requirements for the game platform *Ready Player Pi* (RPPi). The documentation sets the foundation for architecture, design, and further development of the project, it is substituted by relevant models and diagrams.

1. System Description

Ready Player Pi is an expandable multiplayer game-service controlled by one Raspberry Pi serving as the **Server Host**. Whilst connected, Raspberry Pi’s functions as **RPi Hosts** and **RPi Guests**. There will always be one RPi Host. Our game system will only support two-player games, resulting in one **Host Player** and one **Guest Player** per **Game Room**.

- **Server RPi** oversees system data as well as hosting the database for the leaderboard system. This RPi communicates with both Host RPi and Guest RPi, sending information like system state, game information along with assigning roles in both games, which will be defined later.
- **Host RPi** will be assigned to the player choosing “Host Game” in their Ready Player Pi GUI. This RPi will be in charge of choosing a room code, as well as initializing and starting one of the games, whenever the player chooses to do so.
- **Guest RPi** will be assigned to the player choosing “Join Game” in their Ready Player Pi GUI. In the initial stages, this RPi will communicate with the server which will handle establishing a connection to the Host RPi. When this connection is established and the host is ready, the players can start one of two games.

The following figure shows a sketch of the GUI Flow as seen from the perspective of the Host RPi and the Guest RPi.

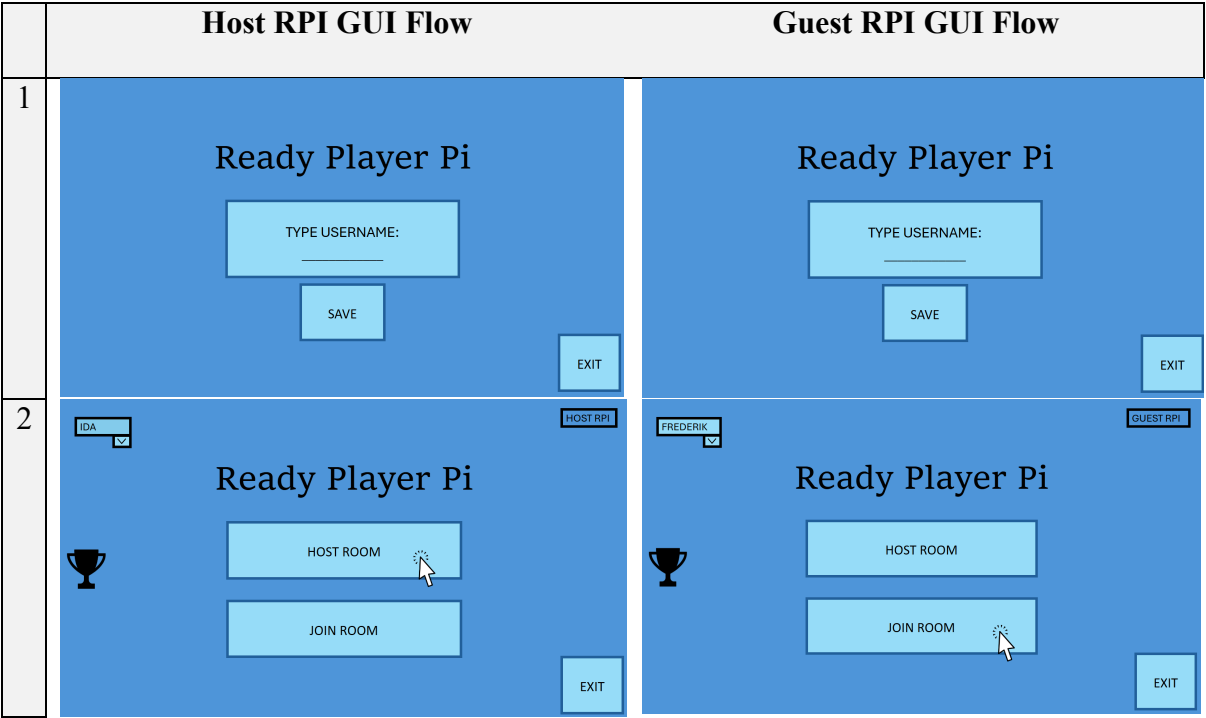




Figure 1 - GUI Flow Sketch

We have chosen to work with two games for our system. Both these games are two player games, which means, each player (Host & Guest) will be randomly assigned either the **Performer** role or the **Communicator** role.

For the *first* game, GyroPi, these players will have the following assignments:

- **Performer Player** will use their RPi along with the ECE-hat (this hat has the gyroscope functionality), to navigate around a map, which can only be seen by the Communicator. The goal is to roll a ball around and communicate with the Communicator, avoiding obstacles and reaching the end.
- **Communicator Player** will be shown a map with obstacles, and a ball that can be controlled by the Performer. The Communicator's job is to manage the Performer to the end by watching the ball's speed and placement on the map.

The following figure shows a sketch of the Gyro-pi Flow as seen from the perspective of the roles: Communicator and Performer.

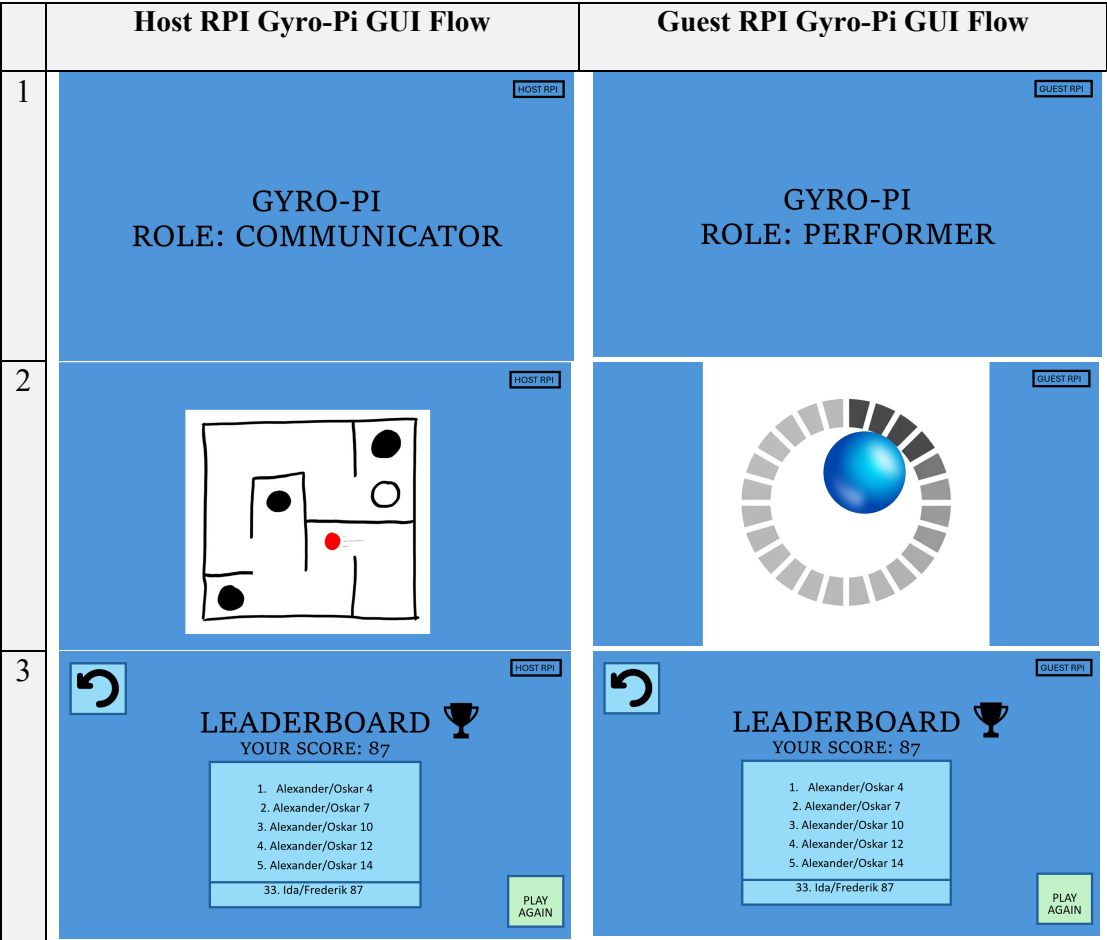


Figure 2 - GUI Flow GyroPi

For the *second* game, NotePi, the players will have the following assignments:

- **Performer Player** will play five notes of music on an instrument, trying to find the right sequence of notes to solve the puzzle presented on the communicators screen. Five correct notes are needed to complete the game. The performers' RPi will pick up the notes using a microphone. The played notes will be sent to the server, which checks whether the notes were the right ones, if they were played in the right order, or if they were completely wrong. The performer will try until succession.
- **Communicator Player** will receive the result of the performers performance from the server presented as a randomly generated Wordle-like puzzle. They must tell the performer which notes are right and wrong. Together, these two players can complete the game.

The following figure shows a sketch of the Note-Pi Flow as seen from the perspective of the roles: Communicator and Performer.



Figure 3 - GUI Flow NotePi

The Host and Guest use a **Game Controller**, which consists of a RPI with all necessary hardware attached, this includes the ECE-hat (gyroscope) for Gyro-Pi and a microphone used in Note-Pi.

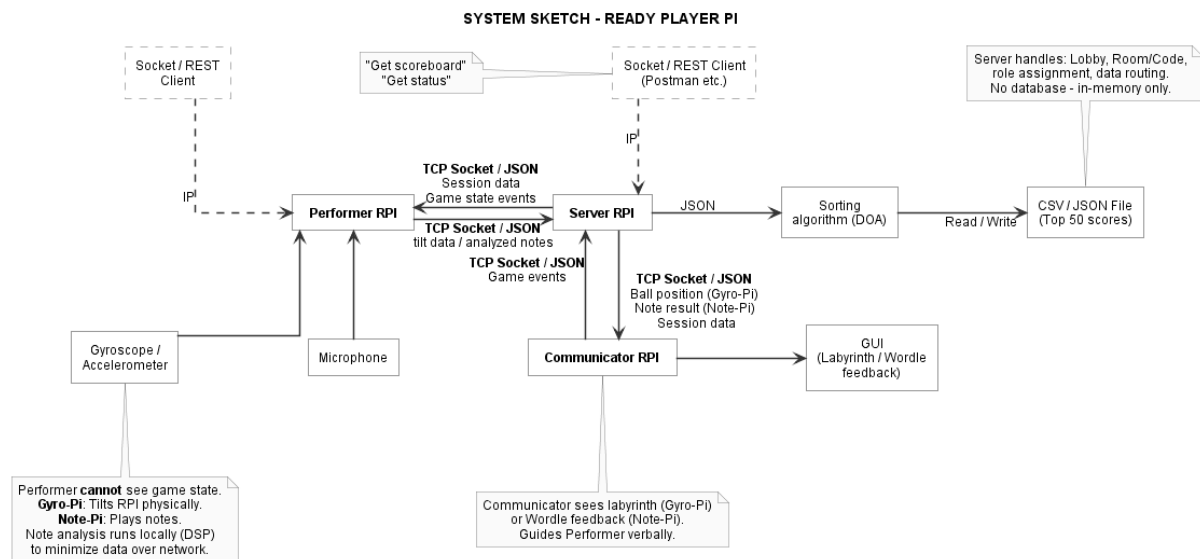


Figure 4 - System Sketch

This system sketch provides a detailed overview of the system within the game loop. It outlines the responsibilities of the Server RPi as well as the two client RPi's. The sketch further details the communication and data transfer between the RPi's and introduces the distinct hardware modules involved. This is an early draft and may change as the project evolves.

2. Functional requirements

The functional requirements are described in this section. To describe these requirements, RPPi is introduced with an Actor-context diagram, used to identify all actors involved. This is supported by a Use Case Diagram, which identifies all Use Cases. These Use Cases are fully dressed, and will define the functional requirements for the system, which are prioritized using MoSCoW.

2.1 Actor-Context Diagram

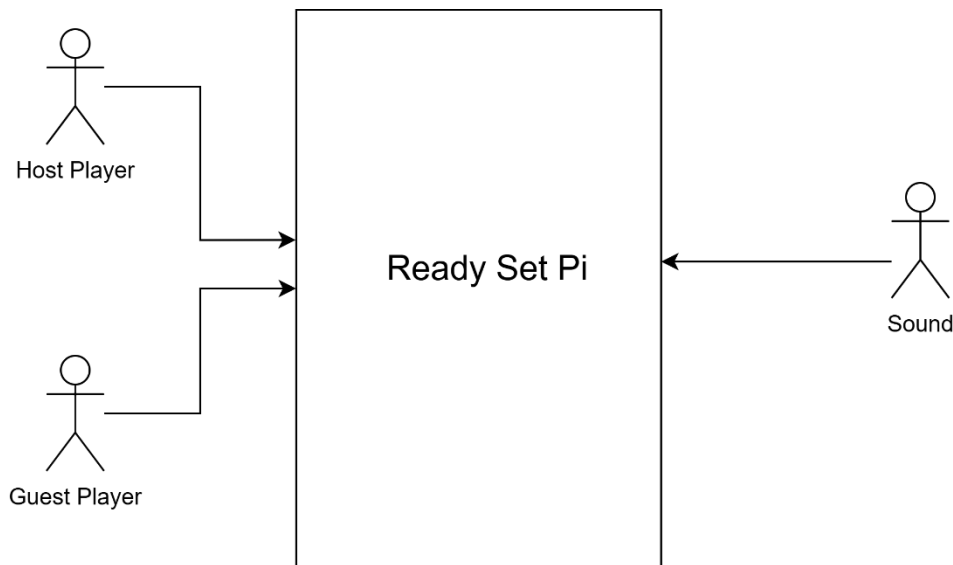


Figure 5 - Actor-Context Diagram

2.2 Actor Descriptions

Name	Player
Type	Primary
Description	A person that uses the application. There are different types of Players.

Name	Host Player
Alternative references	HP
Type	Primary
Description	The host is the actor who initiates the game, creating the instance on the server in which the game is played. The host actor shows the Room pin to the guest(s).

Name	Guest Player
Alternative references	GP

Type	Primary
Description	The player that joins the Room created by Host Player via the Room Pin.

Name	Sound
Alternative references	Notes / Note
Type	Secondary
Description	The game “NotePi” revolves around sounds. The game expects measureable notes, however, any sound will affect the game.

2.3 Use Case Diagram

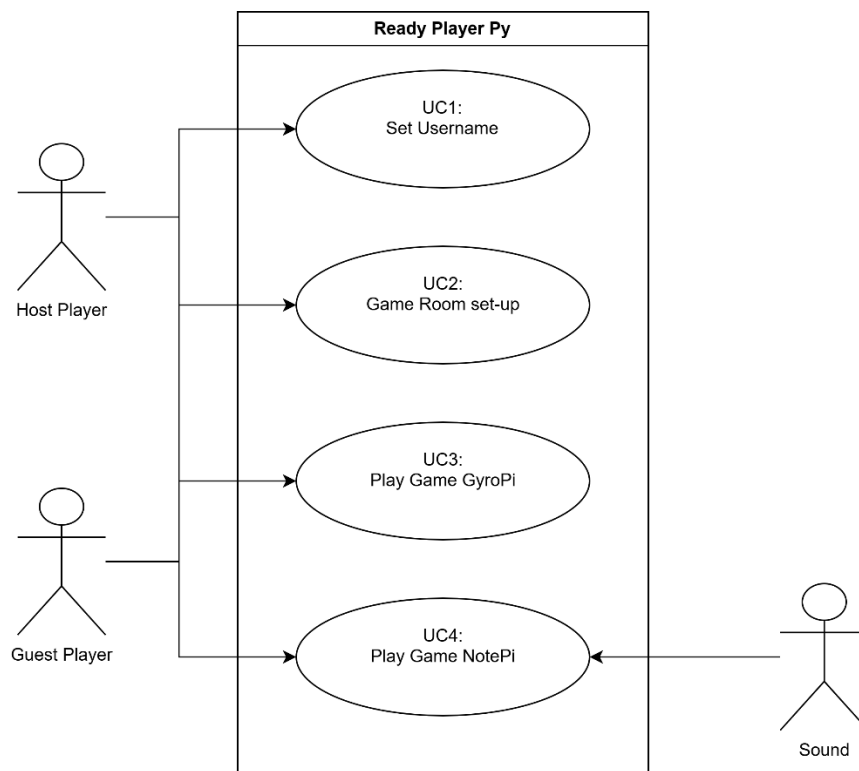


Figure 6 - Use-Case Diagram

2.4 Use Case Descriptions

This section contains fully dressed descriptions of the following use cases:

1. Set Username
2. Game Room set-up
3. Play Game GyroPi
4. Play Game NotePi

2.5 Use Case 1: Set Username

Name:	UC-1: Set Username
Goal:	The Player's Username is saved on their RPi.
Initialization:	The Player runs the application.
Actors:	Player
Precondition:	A Username has not yet been set on the Player's RPi .
Postcondition:	A Username is set on the Player's RPi. The Player's RPi is connected to the server.
Main scenario:	1. The Player runs the application. 2. System detects that Username has not been set. 3. The Player is prompted to type in a username in an input box. - If the username does not exist. The Username is set and saved in a "credentials" file on the Player's RPi. Otherwise the user is prompted to choose another username. 4. Player types username 5. Player clicks 'save'. 6. [Does system accept any name?] 7. Player is forwarded to the main menu
Extensions:	[Ext. 1: Player clicks 'exit' before saving a Username.] [1.a] A Username is not saved in credentials file. [Ext. 2: Player types an invalid username] [2.a] Input box is cleared. [2.b] After Player clicks 'save', RPi displays a message that the typed username is invalid and why it was invalid (invalid character or exceeds character limit). [3.b] Player types a new user name.

2.6 Use Case 2: Game Room set-up

Name:	UC-2: 2. Game Room set-up
Goal:	Host Player and Guest Player are navigated to the Game menu inside Game Room.
Initialization:	Both players are connected to the Room.
Actors:	Host- and Guest player.

Precondition:	A Username is set on both Player's RPi. Both players have opened main menu,
Postcondition:	A game room has been successfully created and both players are connected and ready to start a game.
Main scenario:	1. From main menu Player clicks "Host Room" from menu, becoming the "Host Player". 2. Host Player is forwarded to the Room where the Room Code is presented. 3. The remaining player, Guest, chooses "Join Room" and is forwarded to a sub menu, prompting the player to type in the Room Code of the desired Room. 4. System checks if the code matches any existing room. 5. Guest Player is forwarded to another menu, waiting for the Host Player to click 'Start'. 6. Host Player clicks 'Play'. 7. Host Player and Guest Player are forwarded to a game menu that displays the games they can choose between.
Extensions:	[Ext. 1: The typed-in Room Code does not match any existing Room.] [1.a] The input box is emptied and an error message is displayed on the joining player's screen.

2.7 Use Case 3: Play Game GyroPi

Name:	UC-3: Play Game GyroPi
Goal:	Players win the game. On both players screens are displayed their score, the global scoreboard for the GyroPi game as well as their position on the scoreboard.
Initialization:	Host Player clicks on the GyroPi game.
Actors:	Host- and Guest Player. Communicator- and Performer Player.
Precondition:	Host Player and Guest Player are navigated to the Game Menu inside Game Room.
Postcondition:	The Players' score is saved on server.
Main scenario:	1. Host Player clicks on the GyroPi game. 2. Server randomly assigns the roles of Communicator Player and Performer Player between Host Player and Quest Player.

	3. Both Communicator RPi and Performer RPi displays for a few seconds a message that tell each Player what role they have been assigned as well as the name of the game. 4. Communicator RPi displays a course which has a ball at a starting point, a goal area and black holes. 5. Performer RPi displays an instruction to “tilt” their RPi. 6. Performer tilts RPi and forwards data to the server. 7. Performers RPi displays a graphic that indicates to Performer Player, how their tilting is being registered by the system (i.e. in which direction the ball goes). 8. On Communicator RPi, the ball moves according to tilting of Performer RPi. 9. Communicator tells Performer how to tilt the RPi in order for the ball to move on to the goal area. 10. When ball is located on top of goal area, the following is displayed on both Players’ screen: a. Global scoreboard for GyroPi b. The Game Score for the completed game as well as position on the Score Board.
Extensions:	[Ext. 1: The ball is located on top of a black hole.] [1.a] Communicator RPi and Performer RPi both display a message that says the ball fell into a black hole. [1.b] The ball is moved back to starting point. [Ext. 2: Any Player clicks 'Exit] [2.a] Players are sent back to Game menu inside Game Room.

2.8 Use Case 4: Play Note Game “NotePi”

Name:	UC-4: Play Game NotePi
Goal:	Players win the game. On both players screens are displayed their score, the global scoreboard for the GyroPi game as well as their position on the scoreboard.
Initialization:	Host Player clicks on the NotePi game in game menu.
Actors:	Host- and Guest Player. Communicator- and Performer Player.
Precondition:	Host Player and Guest Player are navigated to the Game Menu inside Game Room.
Postcondition:	The Players’ score is saved on server.
Main scenario:	1. Host Player clicks on the NotePi game in game menu.

	2. Server randomly assigns the roles of Communicator Player and Performer Player between Host Player and Quest Player. 3. Both Communicator RPi and Performer RPi displays for a few seconds a message that tell each Player what role they have been assigned as well as the name of the game. 4. Performer RPi displays message that says to play notes. 5. Performer Player plays a note (e.g. on a piano app on phone), and a microphone records the sound. 6. Performer RPi displays feedback which indicates that the note has been classified¹. 7. Performer Player keeps playing notes until Performer RPi has classified 5 notes. 8. Communicator RPi displays result of the note sequence verification.² 9. Communicator Player tells Performer Player the result. 10. Performer Player is allowed to try playing a note sequence repeatedly. 11. When Performer Player plays the correct note sequence, the following is displayed on both Players' screen: ○ Global scoreboard for NotePi ○ The Game Score for the completed game as well as position on the Score Board.
Extensions:	[Ext. 1: The note was not possible to classify.] [1.a] A message will be displayed to the user to try again until it is able to classify the note. [Ext. 2: Any Player clicks 'Exit.] [2.a] Players are sent back to Game menu inside Game Room. [Ext. 3: No sound detected.] [3.a] A message will be displayed to the user that no <i>sound</i> whatsoever was detected.

¹ Performer RPi has determined which note has been played, e.g. a C or an A#.

² Communicator RPi shows each note as either green (= correct note, correct position), yellow (= correct note, wrong position) or red (= note is not in the sequence at all).

3. Project Scope

The project scope for RPPi is defined using the MoSCoW model. This model categorizes requirements into four groups: Must Have (M), Should Have (S), Could Have (C), Won't Have (W). All functional requirements are categorized into these four groups and have been identified and analyzed based on the use cases and the project description.

ID	Requirements	Source
Must have		
M1	The player must be able to input and save a username in a local credentials file on the RPi.	UC-1
M2	The system must be able to detect if a username is already set on the connected RPis.	UC-1
M3	The RPi must be able to connect to the server after a username is set.	UC-1
M4	The player must be able to select "Host Room" and become a host player with a generated Room Code.	UC-2
M5	A player must be able to select "Join Room" and input a Room Code.	UC-2
M6	The system must be able to validate whether the entered Room Code match an existing room.	UC-2
M7	Both players must be redirected to the game menu with available games.	UC-2
M8	The server must be able to assign Communicator and Performer roles randomly during game start.	UC-3, UC-4
M9	Both RPi's must be able to show a message with the assigned role and the name of the game for a few seconds during game start.	UC-3, UC-4
M10	The RPi with the Communicator role must be able to show a course with a ball at a starting point, a target area and black holes for the GyroPi game.	UC-3
M11	The RPi with the Performer role must be able to register tilting using the gyroscope and send the data to the server.	UC-3
M12	The ball on the Communicator RPi must be able to move according to the Performers tilting.	UC-3
M13	If the ball hits a black hole, the Communicator RPi should have a message shown on the display and the ball should move back to the starting point.	UC-3 Ext.1
M14	The RPi with the Performer role must be able to use a microphone to record and classify notes for the NotePi game.	UC-4
M15	The system must wait for 5 notes to be classified before the sequence can be verified.	UC-4
M16	The communicator RPi must show the result of a note sequence verification like a Wordle like feedback.	UC-4
M17	The Performer Player must be able to try a new sequence continuously.	UC-4

M18	The system must be able to classify notes C4-B4.	UC-4
M19	The server must be able to route data between the Communicator RPi and the Performer RPi	UC-3, UC-4
M20	The Players score must be saved on the server.	UC-3, UC-4
Should Have		
S1	The Performer RPi should have a graphical element that indicates in which <i>direction</i> tilting is registered.	UC-3
S2	The graphical interface mentioned in S1 should also indicate to what <i>degree</i> the tilting is registered, i.e. what pace is registered.	
S3	The Performer RPi should be able to show feedback that indicate when a note has been classified.	UC-4
S4	When a wrong Room Code has been entered the input field should be emptied and a error message shown.	UC-2
S5	Guest Player should see a waiting screen until the Host Player clicks 'Start'.	UC-2
S6	If a note cannot be classified, then a message should be shown to the user prompting the person to try again.	UC-4 Ext. 1
S7	If no sound is detected, a message should be displayed that no sound is currently registered.	UC-4 Ext. 3
S8	The server should maintain a global top 50 leaderboard per game with usernames and their corresponding scores.	UC-3, UC-4
S9	The scoreboard should be sorted and saved in a proper format such as CSV or JSON.	System Sketch
S10	The system should be able to classify all notes on a standard piano, i.e. from A0 to C8.	UC-4
Could Have		
C1	A player could click 'Exit' during gameplay and be send back to the game menu.	UC-3, UC-4
C2	A player could click 'Exit' under username setup without saving a username.	UC-1
C3	Both player positions on the leaderboard could be shown after the game ended.	UC-3, UC-4
C4	The Performer RPi could show an instruction to 'tilt' or 'play note' depending on the game.	UC-3, UC-4
C5	A Socket/REST Client such as a Postman instance could be able to fetch scoreboard and status from the server.	System Sketch
Won't Have		
W1	More than two players per Game Room.	UC-2
W2	Database for persistence of leaderboard.	System Sketch
W3	Automatically reconnect when a connection has been lost.	Implicit
W4	More than two games in the game menu.	UC-2

4. Non-Functional Requirements

The table below lists the non-functional requirements for the system. The non-functional requirements are categorized according to the -URPS (Usability, Reliability, Performance and Supportability) from the FURPS-model.

FURPS Category	ID	
Usability	NF1	Wireless connection between server and clients.
	NF2	Game is displayed in English.
	NF3	User manual for each game accessible in the game room, marked by an 'I'.
	NF4	Menus and Games are in colors.
Reliability	NF5	Connection status is displayed for each player.
	NF6	The system notifies the player on disconnection.
Performance	NF7	When Host player clicks 'Host Room', a Room Code should be displayed within 5 seconds.
	NF8	When Guest Player has typed Room Code, the Guest RPi should respond within 5 seconds.
	NF9	At Game start, the screen displays the assigned roll (Communicator/Performer Player) for 1-5 seconds.
	NF10	Gyro-Pi Game: Whenever the Performer Player tilts the game controller, the Observer Player's game controller will register the motion within 0.05 seconds.
	NF11	Gyro-Pi Game: If the ball hits a black hole, the ball is moved back to starting position in less than 2 seconds.
	NF12	Note-Pi Game: Latency from when a note is played to it being registered is less than 2 seconds.
	NF13	Note-Pi Game: In 9/10 instances, the note will be correctly classified.
	NF14	Note-Pi Game: Maximum 5 seconds after Communicator has received results from a note sequence that Performer Player has just played, the system is able to listen and classify a new sequence of notes.
	NF15	The global scoreboard shall be updated and sorted within 1 second after game completion.
Supportability / Safety		The system architecture will be modular to allow future expansion, i.e. more Games.
		The codebase shall follow consistent naming conventions and documentation standards.
	NF16	The username may only include lower- and uppercase letters as well as numbers.

	NF17	Username cannot exceed 12 characters.
--	-------------	---------------------------------------

Noter:

- The system should be able to classify all notes on a standard piano – i.e. from A0 to C8.